03

MGU BCA - 3RD SEM

Feature Engineering

Module 3 - Lab Practice

By Aben Sojan

Exam - 18 Nov 2026

Complete question bank: every important, repeated and sure-mark question for this module, each with a full-mark model answer. Nothing important left out.

EXAM PATTERN (from real MGU question papers)

FE theory exam: 35 marks, 1 hour (MG3DSEBCA201 Honours model paper, mgu.ac.in Oct 2025).

Printed: 10 x Part A (2 marks) + 5 x Part B (5 marks) = 45 marks printed, so there IS choice.

Likely: Part A all compulsory (20) + answer any 3 of 5 in Part B (15) = 35. Exact choice rule UNVERIFIED - confirm from the paper.

Module 3 (lab: numpy, pandas, sklearn, EDA, log transform, text) has ZERO questions in the model paper and none in related papers.

M3 is almost certainly assessed through the PRACTICAL exam, not theory. Study it for practicals; theory marks come from M1 and M2.

TOPIC 1

1

Python Libraries for ML

numpy: fast numerical arrays and maths operations.

pandas: DataFrames for loading, cleaning and exploring tabular data.

sklearn: ready-made ML models, preprocessing tools and metrics.

matplotlib and seaborn: plots and charts; nltk: text processing; Kaggle: ready datasets.

QUESTION BANK

1 List the Python libraries used in ML and their purposes.

IMPORTANT

Python dominates ML because its libraries cover every step of the pipeline.

numpy provides fast numerical arrays and maths operations; it is the foundation under almost everything.

pandas provides DataFrames for loading, cleaning and exploring tabular data with simple commands.

sklearn offers ready-made ML models plus preprocessing tools like scalers, encoders and metrics.

matplotlib is the basic plotting library, while seaborn builds prettier statistical plots on top of it.

nltk handles text processing with tokenizers, stopword lists and stemmers.

Kaggle is not a library but the main source of ready datasets for practice.

In short, numpy computes, pandas organises, sklearn models, and matplotlib shows.

2 What is the difference between matplotlib and seaborn?

IMPORTANT

Both draw plots in Python, but they sit at different levels of convenience.

matplotlib is the basic low-level plotting library; you control every detail, but simple plots need several lines.

seaborn is built on top of matplotlib and is designed for statistical plots with beautiful defaults.

Example: sns.heatmap() draws a full correlation matrix heatmap in a single line.

seaborn understands pandas DataFrames directly, so plotting a column is effortless.

Use matplotlib when you need total custom control; use seaborn for quick attractive exploratory plots.

Most data scientists use both: seaborn for speed, matplotlib for fine-tuning.

In short, seaborn is matplotlib with better looks and less effort.

3 What is nltk used for in feature engineering?

IMPORTANT

nltk, the Natural Language Toolkit, is the standard Python library for text preprocessing.

It provides tokenizers that split sentences into words.

It provides stopword lists for removing common words like the and is.

It provides stemmers and lemmatizers that reduce words to their base forms.

Example: a product review is tokenized, cleaned of stopwords and stemmed before being turned into features.

These cleaned words then become features through methods like bag-of-words or TF-IDF.

Without nltk-style cleaning, text features stay noisy and models perform poorly.

In short, nltk turns messy raw text into clean tokens ready for feature creation.

TOPIC 2

Missing Data Imputation - Lab

2

pandas finds missing values with isnull() and counts them with sum().

Fill with mean for normal numeric data, median for skewed data, mode for categories.

Drop rows with dropna() only when very few rows are affected.

sklearn SimpleImputer can do the same filling inside a pipeline.

QUESTION BANK

4 How do you find and count missing values in pandas?

IMPORTANT

The first step of handling missing data is finding where the gaps are.

isnull() returns a True/False table marking every missing cell as True.

Chaining sum() counts the True values, giving the missing count per column.

Code:

```
df.isnull().sum()
```

This prints one number per column, instantly showing which columns have gaps.

For a quick overall check, df.isnull().sum().sum() gives the total missing cells.

Always run this before deciding whether to fill or drop.

In short, isnull().sum() is the standard missing-value detector in pandas.

5 Write pandas code to fill missing values with mean, median and mode.

IMPORTANT

The fillna() method replaces missing values, and the strategy depends on the column type.

Fill a numeric column with its mean:

```
df["age"] = df["age"].fillna(df["age"].mean())
```

Use the median for skewed numeric data, where the mean would mislead:

```
df["salary"] = df["salary"].fillna(df["salary"].median())
```

Use the mode for categorical columns, picking the most frequent value:

```
df["city"] = df["city"].fillna(df["city"].mode()[0])
```

The [0] is needed because mode() returns a Series even for a single value.

Choose the strategy by data type: mean for normal numbers, median for skewed numbers, mode for categories.

In short, fillna with the right statistic keeps the column honest.

6 When do you use median instead of mean for imputation?

IMPORTANT

Use the median instead of the mean whenever the data is skewed or contains outliers.

The mean is pulled toward extreme values, so it stops representing the typical value.

Example: salaries 30000, 32000, 35000 and 5000000 have a mean near 13 lakhs, which describes nobody.

The median is the middle value, so extreme values cannot drag it around.

In that salary example the median is around 33500, a far more sensible fill value.

Filling with a distorted mean would inject wrong information into every blank cell.

The same logic applies to house prices, incomes and any long-tailed data.

In short, use the median whenever outliers or skew make the mean untrustworthy.

7 Write sklearn code for missing-value imputation.

IMPORTANT

sklearn fills missing values with SimpleImputer, which fits neatly inside ML pipelines.

Unlike pandas fillna, it learns the fill values on training data and reuses them on test data.

Code for mean imputation:

```
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy="mean")
X_filled = imp.fit_transform(X)
```

The strategy can be mean, median, most_frequent or constant.

For a constant fill, add fill_value, like SimpleImputer(strategy="constant", fill_value=0).

Using it in a pipeline prevents data leakage between train and test sets.

In short, SimpleImputer is the pipeline-friendly way to fill gaps.

TOPIC 3

Data Cleaning - Lab

3

Remove duplicate rows with drop_duplicates().

Fix wrong data types with astype(), like text numbers to int.

Strip extra spaces and fix case with str.strip() and str.lower().

Handle outliers by clipping or removing with the IQR method.

8 Write pandas code to remove duplicates and fix data types.

IMPORTANT

Cleaning starts with removing duplicates and correcting column types.

Remove duplicate rows so repeated records do not get extra weight:

```
df = df.drop_duplicates()
```

Convert wrongly typed columns, like age stored as text, to the right type:

```
df["age"] = df["age"].astype(int)
```

Clean messy text columns by stripping spaces and unifying case:

```
df["name"] = df["name"].str.strip().str.lower()
```

After stripping, " Male " and "male" become the same value.

Always check dtypes with df.dtypes after conversion to confirm the fix.

These three lines solve the most common cleaning problems in one pass.

9 How do you detect and remove outliers using the IQR method?

IMPORTANT

Definition: The IQR (interquartile range) method finds outliers using the spread of the middle 50 percent of data.

IQR equals Q3 minus Q1, the third quartile minus the first quartile.

Any value below Q1 - 1.5*IQR or above Q3 + 1.5*IQR is treated as an outlier.

Code:

```
q1 = df["age"].quantile(0.25)
q3 = df["age"].quantile(0.75)
iqr = q3 - q1
df = df[(df["age"] >= q1 - 1.5*iqr) & (df["age"] <= q3 + 1.5*iqr)]
```

This keeps only the rows inside the acceptable range.

The IQR method is robust because quartiles, unlike mean and standard deviation, resist outliers themselves.

In short, quartiles define normal, and anything far outside is an outlier.

TOPIC 4

4

EDA - Histograms, Boxplots, Correlation Matrix

EDA (Exploratory Data Analysis) means understanding data with plots and stats before modeling.

Histograms show the shape of one numeric column; boxplots show median, quartiles and outliers.

A correlation matrix (heatmap) shows how features relate to each other.

Always do EDA first: it reveals missing values, skew and wrong data.

QUESTION BANK

10 What is EDA? Why is it done before modeling?

IMPORTANT

Definition: EDA (Exploratory Data Analysis) is the process of understanding data with statistics and plots before building a model.

It reveals missing values, outliers, skewed distributions and wrong data types early.

It shows how features relate to each other and to the target, guiding feature selection.

Skipping EDA means the model may silently learn from garbage data.

Example: EDA might reveal that 40 percent of ages are missing, which changes the whole cleaning plan.

It also suggests which features need scaling, encoding or transformation.

The common EDA toolkit is describe(), histograms, boxplots and a correlation heatmap.

In short, EDA is done first because you must understand the data before you can model it.

11 What does a boxplot show? Explain its parts.

IMPORTANT

Definition: A boxplot is a compact visual summary of a numeric column distribution.

The box stretches from Q1 (25th percentile) to Q3 (75th percentile), showing the middle 50 percent of data.

A line inside the box marks the median, the middle value.

The whiskers extend to the normal range of data on both sides.

Dots beyond the whiskers are outliers, values far from the rest.

It quickly reveals spread, skew and extreme values of a column at one glance.

Comparing boxplots across categories, like salary by department, instantly shows group differences.

In short, a boxplot packs quartiles, median and outliers into one small picture.

12 What is a correlation matrix? How is it read?

IMPORTANT

Definition: A correlation matrix is a table showing the correlation value between every pair of features.

Each cell holds a number between -1 and +1 measuring how two features move together.

Values near +1 mean strong positive relation: both rise together, like study hours and marks.

Values near -1 mean strong negative relation: one rises as the other falls.

Values near 0 mean weak or no linear relation.

It is read as a heatmap, where dark or intense cells highlight the strongest relations.

It helps drop redundant features: two features with 0.95 correlation carry nearly the same information.

In short, the correlation matrix maps the relationships hiding in the data.

13 Write code for a histogram, boxplot and correlation heatmap.

IMPORTANT

These three plots form the core visual toolkit of EDA.

Histogram of one column shows its distribution shape:

```
df["age"].hist()
```

Boxplot of one column shows median, quartiles and outliers:

```
df.boxplot(column="age")
```

Correlation heatmap of all numeric columns shows every pairwise relation:

```
import seaborn as sns
sns.heatmap(df.corr(), annot=True)
```

The annot=True flag prints the actual correlation numbers inside the cells.

Run these three right after loading any new dataset.

In short, hist for shape, boxplot for spread, heatmap for relations.

14 List the steps of EDA in order.

IMPORTANT

EDA follows a fixed order so that nothing important is missed.

Step 1: inspect the basics with shape, info() and describe() to see size, types and summary stats.

Step 2: find missing values with isnull().sum() and decide how to handle them.

Step 3: plot distributions with histograms and boxplots to see shape, skew and outliers.

Step 4: check relationships with a correlation heatmap and scatter plots against the target.

Step 5: note every issue found: what needs cleaning, scaling, encoding or transformation.

Only after these steps should modeling begin.

In short: inspect, find gaps, plot shapes, check relations, then plan the fixes.

TOPIC 5

Binning and Discretization - Lab

5

pandas cut() makes equal-width bins; qcut() makes equal-frequency bins.

Labels can be given to bins, like Young, Adult, Senior.

Binning is often followed by one-hot encoding of the bins.

Check bin counts after cutting to confirm the split is sensible.

QUESTION BANK

15 Write pandas code for equal-width and equal-frequency binning.

IMPORTANT

pandas offers two one-line functions for binning.

cut() makes equal-width bins with the same interval size:

```
df["age_group"] = pd.cut(df["age"], bins=4)
```

This splits the age range into 4 equal intervals automatically.

qcut() makes equal-frequency bins with the same record count each:

```
df["age_group"] = pd.qcut(df["age"], q=4)
```

Custom ranges and meaningful labels can be supplied to cut():

```
pd.cut(df["age"], bins=[0, 18, 35, 60, 100], labels=["Child", "Youth", "Adult", "Senior"])
```

After binning, check value_counts() to confirm the split looks sensible.

In short, cut() for fixed ranges and qcut() for balanced groups.

16 When would you use cut() instead of qcut()?

IMPORTANT

Use cut() when the bin ranges themselves carry meaning and should stay fixed.

Example: age groups 0-18, 18-35, 35-60 make business sense regardless of data spread.

Use qcut() when you want balanced groups, especially with skewed data.

Example: income is heavily skewed, so qcut() gives four groups with equal customer counts.

cut() on skewed data can leave some bins nearly empty, wasting the grouping.

qcut() on uniform data can create odd-looking ranges, since it forces equal counts.

The choice depends on whether meaningful ranges or balanced groups matter more.

In short, cut() for meaningful fixed ranges and qcut() for balanced skewed data.

17 Why are labels given to bins? Give an example.

IMPORTANT

Labels turn numeric bin ranges into meaningful names that humans understand.

Raw bins like (18, 35] mean little in a report, but the label Youth is instantly clear.

Example: bins [0, 18, 60, 100] labeled Child, Adult, Senior.

Labeled bins make plots, tables and presentations far easier to read.

They also make the feature self-documenting for anyone using the model later.

In pandas, labels are passed directly: pd.cut(..., labels=["Child", "Adult", "Senior"]).

The number of labels must match the number of bins exactly.

In short, labels translate bin math into human language.

TOPIC 6

Polynomial and Interaction Features - Lab

6

sklearn PolynomialFeatures creates polynomial and interaction terms automatically.

Example: degree=2 on [x1, x2] gives x1, x2, x1^2, x2^2, x1*x2.

Use include_bias=False to skip the constant column.

Scale features before generating polynomial terms.

QUESTION BANK

18 Write sklearn code to create polynomial features of degree 2.

IMPORTANT

PolynomialFeatures generates polynomial and interaction terms automatically.

Code:

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2, include_bias=False)
X_new = poly.fit_transform(X)
```

On features [x1, x2] this creates x1, x2, x1^2, x2^2 and x1*x2.

include_bias=False skips the constant column of ones, which most models add themselves.

Always scale features before this step, since squared large values explode in size.

Use poly.get_feature_names_out() to see the names of the generated columns.

In short, three lines turn straight-line features into curve-ready ones.

19 How do you create only interaction features in sklearn?

IMPORTANT

Sometimes you want feature combinations without the squared terms.

PolynomialFeatures has a dedicated mode for exactly this.

Code:

```
poly = PolynomialFeatures(degree=2, interaction_only=True, include_bias=False)
X_new = poly.fit_transform(X)
```

On [x1, x2, x3] this creates only x1*x2, x1*x3 and x2*x3, skipping x1^2 and friends.

This is useful when you suspect teamwork between features but not curved single-feature effects.

It also keeps the feature count lower than full polynomial expansion.

In short, interaction_only=True gives products without powers.

TOPIC 7

Log Transformation for Skewed Data

7

Skewed data has a long tail on one side; log transform pulls the tail in.

Apply np.log1p(), which handles zeros safely as log(1+x).

After transform, check the histogram again to confirm it looks normal.

Useful for income, price and population data.

QUESTION BANK

20 What is skewed data? How does log transformation fix it?

IMPORTANT

Definition: Skewed data piles up on one side with a long tail stretching to the other, like incomes with a few very high earners.

Most values sit in a small region while the tail holds extreme values.

This hurts models that assume roughly balanced data, because the tail dominates.

Log transformation compresses large values much more than small ones.

After the transform, the long tail shrinks and the distribution looks far more normal.

Example: incomes 30000, 35000, 40000 and 5000000 become much more comparable on the log scale.

The model can then learn the real pattern instead of chasing the tail.

In short, log transform tames the tail and balances skewed data.

21 Why is log1p preferred over log? Write the code.

IMPORTANT

The plain log function has a fatal weakness: log(0) is undefined and crashes the code.

Real data often contains zeros, like zero purchases or zero rainfall days.

log1p computes log(1+x) instead, so zero safely becomes log(1) = 0.

For positive values it behaves almost exactly like log, just shifted.

Code:

```
import numpy as np
df["log_income"] = np.log1p(df["income"])
```

This one line safely transforms the whole column, zeros included.

In short, log1p is log without the zero crash.

22 **Write code to apply log transformation and plot before/after.**

IMPORTANT

Always verify a transform visually by comparing before and after plots.

Code:

```
import numpy as np
df["income"].hist()
np.log1p(df["income"]).hist()
```

The first plot shows the original skewed shape with its long tail.

The second plot should look more bell-shaped and balanced.

If the tail is still long, a stronger approach like Box-Cox may be needed.

This before/after check takes seconds and confirms the transform actually helped.

In short, plot, transform, plot again.

TOPIC 8

8

Text Data Preprocessing

Steps: lowercase, remove punctuation and numbers, tokenize, remove stopwords, stem or lemmatize.

Tokenization splits text into words; stopwords are common words like the and is.

Stemming chops word endings (running -> run); lemmatization gives real dictionary words.

nltk provides tokenizers, stopword lists and stemmers.

QUESTION BANK

23 **Explain the steps of text preprocessing.**

IMPORTANT

Definition: Text preprocessing cleans raw text so it can become reliable features.

Step 1: lowercase everything, so Good and good count as the same word.

Step 2: remove punctuation, numbers and extra spaces that carry no meaning.

Step 3: tokenize, splitting the text into individual words.

Step 4: remove stopwords, the very common words like the and is.

Step 5: stem or lemmatize the remaining words to their base forms.

After these steps the text is a clean list of meaningful tokens.

These tokens then become features through bag-of-words or TF-IDF.

24 Distinguish between stemming and lemmatization.

IMPORTANT

Both reduce words to base forms, but they differ in method and quality.

Stemming chops word endings using fixed rules, which is fast but crude.

Example: studies becomes studi, which is not even a real word.

Lemmatization uses a dictionary and grammar rules to find the true base form.

Example: studies becomes study and better becomes good.

Lemmatization is slower but far more accurate than stemming.

Use stemming for speed on huge data and lemmatization when quality matters.

In short, stemming cuts blindly while lemmatization thinks.

25 What are stopwords? Why are they removed?

IMPORTANT

Definition: Stopwords are very common words like the, is, and and in.

They appear in almost every document and carry little distinctive meaning.

Keeping them wastes memory and slows down feature creation.

Worse, they dilute the signal: a spam filter learns nothing from the word the.

Removing them shrinks the vocabulary and helps the model focus on important words.

nltk provides ready stopword lists for many languages.

Some tasks keep negations like not, since not good means the opposite of good.

In short, stopwords are noise words, and removing them sharpens the text signal.

26 Write nltk code for tokenization and stopword removal.

IMPORTANT

nltk makes tokenization and stopword removal a few lines of code.

Code:

```
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
words = word_tokenize(text.lower())
stops = set(stopwords.words("english"))
clean = [w for w in words if w.isalpha() and w not in stops]
```

word_tokenize splits the text into words; lowercasing unifies the case.

The list comprehension keeps only alphabetic words that are not stopwords.

Note: nltk data (punkt, stopwords) must be downloaded once with nltk.download().

In short, tokenize, filter, and clean text is ready.

TOPIC 9

9

PCA - Lab

Standardize features first with StandardScaler, then apply PCA.

Choose the number of components that keeps most variance, e.g. 95 percent.

explained_variance_ratio_ tells how much info each component keeps.

Reduced data can be plotted or fed into a model.

QUESTION BANK

27 **Write complete sklearn code for PCA keeping 95% variance.**

IMPORTANT

This is the standard PCA pipeline: scale first, then reduce.

Code:

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
X_scaled = StandardScaler().fit_transform(X)
X_pca = PCA(n_components=0.95).fit_transform(X_scaled)
```

StandardScaler gives every feature mean 0 and standard deviation 1, which PCA requires.

Passing 0.95 as n_components tells sklearn to keep enough components for 95 percent of variance.

The result X_pca has far fewer columns but keeps 95 percent of the information.

Fit the scaler and PCA on training data only, then transform test data with the same fitted objects.

In short, scale first and let sklearn pick the component count for 95 percent variance.

28 How do you decide how many components to keep in PCA?

IMPORTANT

The number of components is chosen by how much variance you want to keep.

Common targets are 90 or 95 percent of the total variance.

Fit PCA with all components first and inspect explained_variance_ratio_.

Code:

```
pca = PCA().fit(X_scaled)
print(pca.explained_variance_ratio_)
```

Add up the ratios from the largest down until the total crosses your target.

Example: ratios [0.6, 0.25, 0.1, 0.05] mean 2 components keep 85 percent and 3 keep 95 percent.

A scree plot of these ratios shows an elbow where extra components add little.

In short, keep adding components until the kept variance reaches your target.

29 What is explained_variance_ratio_ in sklearn PCA?

IMPORTANT

Definition: explained_variance_ratio_ is an array showing the fraction of total variance held by each principal component.

After fitting PCA, each component gets one number in this array.

Example: [0.6, 0.25, 0.1, 0.05] means the first component holds 60 percent of the information and the second 25 percent.

The values always add up to 1.0 across all components.

It is the main tool for deciding how many components to keep: sum the largest ratios until you reach 90 or 95 percent.

It also reveals the data structure: a steep drop means a few directions dominate.

You can read it right after pca.fit(X_scaled) with no extra computation.

In short, it tells you how much each component is worth keeping.